

SURYAGRID AI

Complete Execution and Agentic Architecture

Frontend, Backend, Agents, Data, ML, DSM, RL, Security, and Pilot Workflow

The consolidated implementation roadmap from the current verified prototype to a controlled, evidence-backed production pilot.

Prepared by: SuryaGrid Project Team

Date: 9 July 2026

Repository baseline: 5bed3cd on main

Primary repository: <https://github.com/DBIT-Bangalore/SuryaGrid>

Team document. No individual authorship is asserted. Implemented claims are tied to repository history, current code, tests, model cards, or recorded verification. Planned claims are marked and gated by their required evidence.

Contents

I	Complete Execution and Agentic Architecture	2
1	Outcome and Scope	2
2	Full Frontend-Backend-Agent Architecture	3
3	Frontend to Backend Connection Matrix	3
4	Agentic Architecture and Agent Inventory	4
5	Data, Models, Policy, and Source Plan	6
6	Security, Reliability, and AI Guardrails	7
7	Detailed Execution Roadmap	8
8	Timeline and Milestones	10
9	Standard Implementation Workflow	10
10	Testing and Acceptance	10
11	Cost and Capacity Planning	11
12	Risks and Honest End State	11
13	Immediate Team Actions	12
14	External Technical References	12

Part I

Complete Execution and Agentic Architecture

1 Outcome and Scope

The target is a production-capable solar decision-support platform that connects weather and station data to forecast models, deterministic energy and DSM calculations, safe agentic reasoning, operator workflows, and eventually validated plant telemetry. The system must answer:

1. How much power can each DG or solar station generate over the selected horizon?

2. How does the forecast compare with schedule, request, demand, and measured actuals?

3. What energy gap, risk band, or regulatory DSM result follows?

4. What should an operator review or do next, with complete provenance and calculation trace?

5. Can a constrained RL policy improve advisory recommendations after real local outcomes exist?

I Dual truth model

Every response has a deterministic truth plane and an optional reasoning plane. The truth plane owns all source data, units, forecasts, formulas, model outputs, and money calculations. The reasoning plane may route tools, summarize, detect inconsistencies, and recommend review actions. It may never overwrite numeric results or invent missing values.

1.1 Four-level product model

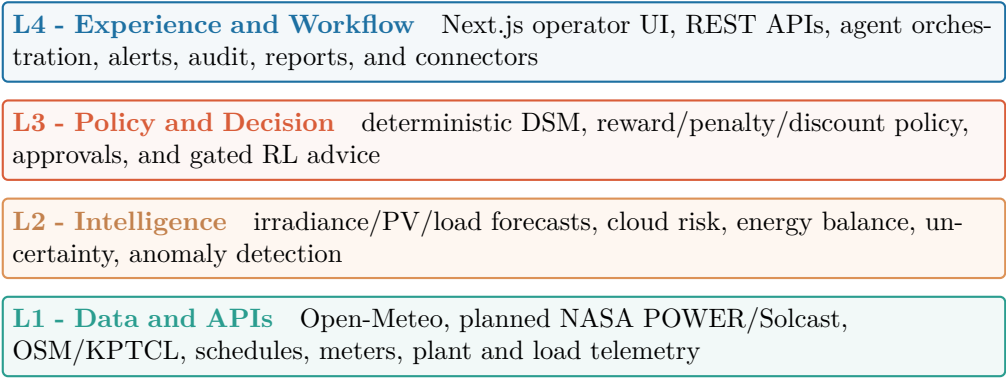


Figure 1: The complete four-level SuryaGrid product model.

2 Full Frontend-Backend-Agent Architecture

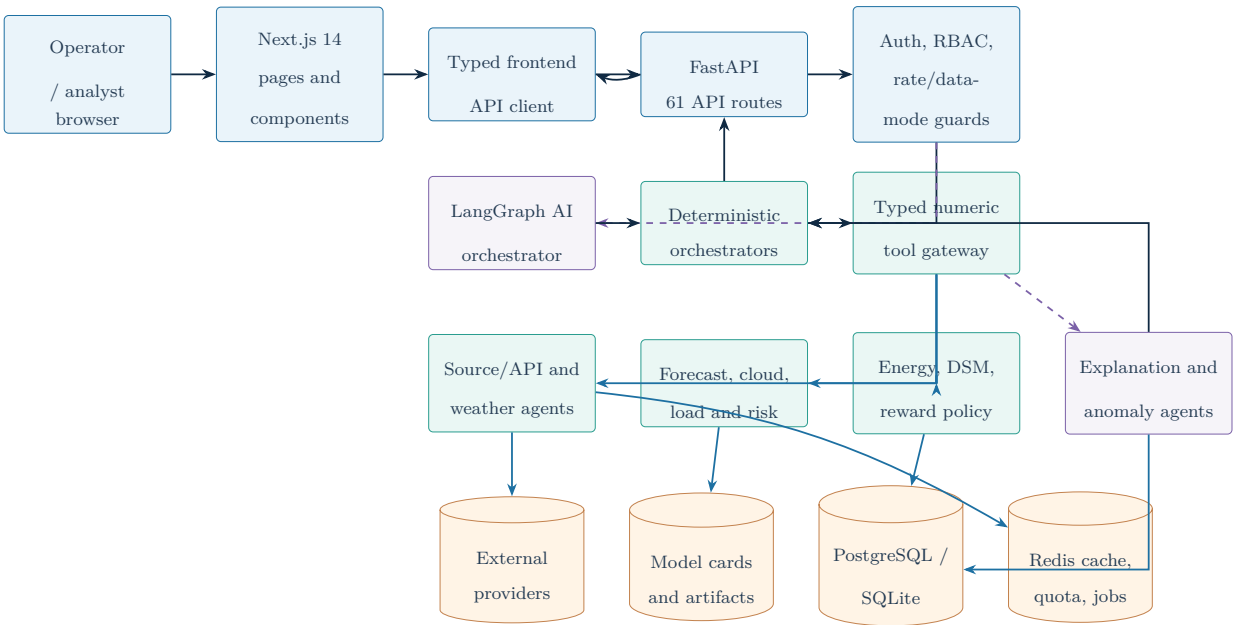


Figure 2: Target connection architecture. Solid navy paths are request/control; blue paths carry typed data; dashed violet paths are optional LLM reasoning.

2.1 Connection contract

1. The UI calls only the typed client in `frontend/lib/api.ts`; pages do not call providers or model files directly.
2. FastAPI validates request identity, role, site scope, data mode, units, and schema.
3. The deterministic orchestrator remains the production fallback and numerical authority.
4. The optional LangGraph route chooses from an allowlist of typed tools; tools call existing providers, models, energy services, and DSM engines.
5. Tool output is immutable and hashed into the trace before an LLM sees it.
6. The LLM returns schema-validated routing, explanation, anomaly, or escalation fields.
7. Persistence stores inputs, source IDs, model/formula/policy versions, outputs, agent trace, calculation trace, latency, cost, and approval status.
8. The API returns one stable response envelope to the UI. If LLM service is unavailable, numeric output still completes and the explanation is marked unavailable or deterministic.

3 Frontend to Backend Connection Matrix

Table 1: Frontend pages, backend contracts, and agent ownership.

Frontend surface	Primary client calls	Backend route groups	Agents/services
Dashboard	health, predict, orchestrate substation, generation timeline	/health, /predict, /orchestrate/substation, /generation/timeline	orchestrator, substation orchestrator, forecast, risk, explanation

Frontend surface	Primary client calls	Backend route groups	Agents/services
Locations	catalog, context, nearest, coverage	/substations/*, /locations/*, /sites/*/data-coverage	location data, source registry, substation context
Predictions	predict site, forecast, model status	/predict/*, /forecast/*, /ml/model/status	forecast, feature engineering, model registry
Timeline	timeline, summary, generation timeline	/timeline/*, /summary/*, /generation/timeline	forecast, energy, DSM classifier
Energy	energy and consumption profiles	/energy/*, /consumption/profiles	energy service, forecast, consumption service
DSM	profiles, advanced check, substation DSM, Karnataka DSM	/dsm/*, /dsm/forecast, /dsm/karnataka/*	DSM classifier, DSM engine, reward/policy
Settlement	settle day, settlement history, RL rates	/settle*, /settlements/*, /rl/rates	reward, persistence, energy/DSM services
RL Lab	train, rates, run history	/rl/train, /rl/rates, /rl/runs	RL environment/trainer, persistence; production use gated
ML	ingest, build, train, predict, status	/ml/*, /kaggle/*	Kaggle data, feature engineering, model registry
Data Sources	source catalog, provider status	/sources/*, /data-sources/status, /providers/status	source registry, API management, live weather
Karnataka	regions, seed, BESCOM status, Karnataka DSM	/karnataka/*, /bescom/status, /dsm/karnataka/*	location, BESCOM connector framework, DSM engine
System	health, system status, agent/data status	/health, /system/status, /agents/*	API management, source registry, all status providers

4 Agentic Architecture and Agent Inventory

4.1 Current deterministic and ML agents

Table 2: Existing agent modules and target role.

Module	Responsibility	Connects to	Status
api_agent.py	provider choice, failover, quota state	providers, Redis, orchestrator	PARTIAL
api_management_agent.py	provider health, retry, rate visibility	system/status routes	PARTIAL
source_registry_agent.py	source records and formula references	registry, provenance, UI	IMPLEMENTED
live_weather_agent.py	live weather and labelled fallback	Open-Meteo, Redis, DB	IMPLEMENTED
kaggle_data_agent.py	Kaggle ingestion and normalization	data pipeline and model builds	IMPLEMENTED
location_data_agent.py	locations, substations, coverage	OSM parquet, site services	IMPLEMENTED
feature_engineering_agent.py	quality checks and model features	dataset builder, ML models	IMPLEMENTED
forecast_agent.py	formula/ML/hybrid solar nowcast	weather, pvlib, model registry	IMPLEMENTED
dsm_classifier_agent.py	interval deviation classification	schedule, forecast/actual	IMPLEMENTED

Module	Responsibility	Connects to	Status
dsm_engine_agent.py	rule-profile resolution and persistence	DSM repository and policy	PARTIAL
risk_agent.py	deterministic operational risk	forecast and DSM output	IMPLEMENTED
fuzzy_risk_agent.py	fuzzy risk inference	forecast/DSM features	IMPLEMENTED
reward_agent.py	reward/penalty/discount research engine	settlement and RL	GATED
explanation_agent.py	deterministic explanation text	final result envelope	IMPLEMENTED
persistence_agent.py	saves important records	DB repository and audit	IMPLEMENTED
orchestrator_agent.py	standard interval sequence	all numeric agents	IMPLEMENTED
substation_orchestrator.py	substation-driven seven-step flow	context, weather, ML, DSM	IMPLEMENTED

4.2 Planned LLM reasoning agents

Table 3: New agentic reasoning layer.

Agent	Allowed responsibility	Forbidden responsibility	Fallback
AI Orchestrator	select approved tools, route around missing inputs, request review, explain routing	changing numeric outputs, tariffs, schedules, or control set-points	deterministic orchestrator
AI Explanation	structured summary, evidence-linked narrative, operator recommendations	inventing causes, values, regulations, or confidence	deterministic explanation
Anomaly Detective	interpret deterministic anomaly flags, group incidents, suggest investigation	being the only detector or suppressing hard-rule alerts	rule-based detector

4.3 LangGraph execution design

Use a typed StateGraph with explicit START/END, conditional edges, durable checkpoints, bounded retries, and interrupt points for human approval. The design is:

1. **Analyze context:** deterministic validation creates allowed/missing/blocked fields.
2. **Choose tools:** AI orchestrator selects only tools whose prerequisites are present.
3. **Execute tools:** numeric tools run outside the model and return typed results.
4. **Validate:** schema, unit, bounds, provenance, and output hash checks run.
5. **Assess anomaly:** hard rules/statistics flag anomalies; LLM adds explanation only.
6. **Apply policy:** deterministic effective-dated DSM/reward profile runs.
7. **Interrupt if required:** unsupported money, low confidence, policy change, or control recommendation requires a human decision.

8. **Synthesize:** explanation agent returns strict JSON fields.
9. **Persist and publish:** state/checkpoint, audit, costs, and final envelope are saved.

4.4 Shared state and communication protocol

Table 4: Minimum shared state contract.

State group	Required fields	Writer
Identity	trace ID, user/role, tenant, station/site, request timestamp	API guard
Context	coordinates, timezone, units, capacity, schedule, policy ID, missing fields	context service
Sources	source IDs, retrieval time, license, raw hash, fallback and quality status	source/API agents
Forecast	GHI/PV/load intervals, confidence, model/formula versions	forecast tools
Decision	measured actual, schedule deviation, energy gap, DSM band/amount, blocked fields	logic and policy tools
Reasoning	route decision, explanation JSON, anomalies, recommended review	LLM agents
Governance	approval, interrupt reason, retry count, token/cost, latency, final status	orchestrator and persistence

Agent-to-agent communication uses state, not free-form hidden conversation. Every node appends an event to `agent_trace`; every numeric result appends source/formula/model details to `calculation_trace`. Errors are typed as retryable, fallback-eligible, review-required, or blocked. No agent may silently convert a blocked result into a value.

5 Data, Models, Policy, and Source Plan

5.1 Provider and data priorities

Table 5: Data and policy integration priority.

Source/work	Current state	Planned implementation	Required gate
Open-Meteo	live weather and archive	retain as primary weather provider	provider SLO and cache
NASA POWER	historical cross-check only	runtime historical/NRT fallback provider, not a day-ahead forecast substitute	latency/freshness test
Solcast	not integrated	optional contracted forecast provider	contract, quota, cost
NPP/Grid India reports	public reports confirmed	feasibility parser for region/state generation and capacity; do not assume hourly Karnataka load until inspected	schema proof

Source/work	Current state	Planned implementation	Required gate
KPTCL/Aikosh candidate	not verified in repo	inspect access, fields, licensing, current vs planned capacity, then match conservatively	source and match review
Local PV actuals	absent	KPCL/BESCOM/operator data agreement and ingestion contract	meter semantics and consent
KERC 2026 DSM	externally reported; not encoded from archived official source	effective-dated cumulative slab engine with source snapshot	official gazette + legal review
Load model	India-wide, non-production locally	train regional then Karnataka/site model	chronological holdout and domain review
RL policy	production blocked	offline constrained research then advisory shadow mode	real outcomes + approved economics

5.2 KERC 2026 handling

Published reports describe a solar tolerance band of 5%, cumulative charges of Rs. 0.25, Rs. 0.50, and Rs. 0.75 per unit across increasing deviation slabs, a scheduled-generation denominator, a deemed 1 MW schedule for zero-schedule injection, and an annual cap of 3 paise per unit times annual generation. These values are **requirements candidates**, not yet permission to emit settlement money.

Before `emits_rupee_values=true`:

1. archive the official Gazette/KERC document and checksum it;
2. record notification date, effective date, applicability, section/table references, amendment status, and legal reviewer;
3. implement cumulative slab arithmetic, annual cap, interval aggregation, zero-schedule handling, over/under-generation treatment, and version selection;
4. independently reproduce golden cases and compare against a manual worksheet;
5. add dual approval, rollback, and audit for rule activation.

6 Security, Reliability, and AI Guardrails

- OIDC/JWT authentication, RBAC, tenant and site isolation on every protected route.
- Secret manager for provider and OpenRouter keys; never expose keys to browser or logs.
- Request rate limits plus separate LLM token, spend, and concurrency budgets.
- Tool allowlists, JSON Schema output, prompt-injection filtering, and no arbitrary code tools.
- Deterministic numeric equivalence test: AI and non-AI routes return identical numeric fields.
- Model/provider pinning with explicit fallback list and recorded final model ID.
- LLM outage, timeout, malformed output, and budget exhaustion fall back deterministically.

- Human interrupts for money activation, rule changes, low-confidence recommendations, external messages, or any future control action.
- Immutable audit: prompt template version, tool inputs/outputs, response schema, token usage, latency, reviewer, and release version.
- HTTPS, backups, restore drills, provider outage tests, SLOs, data drift, model drift, and incident runbooks before pilot.

7 Detailed Execution Roadmap

7.1 Phase 0 - Foundation hardening (Week 1)

Work items

1. Enforce `APP_DATA_MODE` on every API and background job. Real mode forbids silent synthetic fallback; clear-sky physics is labelled separately.
2. Unify the source registry and model-card provenance into one `SourceRecord` contract.
3. Verify Docker Compose end-to-end, health checks, migrations, backend tests, frontend build, and evidence log.
4. Quarantine unsupported monetary defaults and add a test that forbids unapproved currency.

Deliverables: data-mode behavior matrix, unified registry, no-orphan-source tests, Docker verification log, and unchanged 118-test baseline or better.

7.2 Phase 1 - Provider and regulation integration (Weeks 2–3)

NASA POWER

- Add `app/providers/nasa_power.py` behind `WeatherProvider`.
- Map GHI, temperature, humidity, wind, and pressure; use explicit UTC/LST conversion.
- Cache responses, record freshness, and use it for historical fill/NRT fallback.
- Cross-validate against Open-Meteo; never call reanalysis a live forecast.

KERC rule parser

- Add effective-dated `kerc_dsm_2026.py` only after the official-source gate.
- Support cumulative slabs, annual cap, deemed schedule, applicability, and source citations.
- Add scenario tests, manual reproduction, frontend attribution, and rollback.

7.3 Phase 1.5 - Real agentic reasoning layer (Weeks 3–4)

- Add `langgraph`, `langchain-core`, and the selected OpenAI-compatible client.
- Create `app/agents/ai/state.py`, `tools.py`, `orchestrator.py`, `explanation.py`, `anomaly_detective.py`, `prompts`, and `policies`.
- Add `/api/v1/orchestrate/substation/ai` beside the deterministic route.
- Add strict structured output, durable checkpointing, retry limits, and approval interrupts.
- Add `AIInsightsPanel`, `anomaly/alert` UI, and reasoning trace display.
- Prove numeric equivalence and deterministic fallback under LLM failure.

7.4 Phase 2 - Data acquisition and enrichment (Weeks 5–6)

- Inspect NPP/Grid India report files before defining granularity; build fixture-driven parser.
- Obtain regional/Karnataka load from a source that actually exposes required intervals.
- Inspect KPTCL/Aikosh or official KPTCL/CEA capacity sources; distinguish planned augmentation from commissioned capacity.
- Match stations by normalized name, operator, voltage, and geography with manual review.
- Begin local PV data requests to KPCL, BESCOM, and willing plant operators.
- Prepare a local-PV ingestion schema for timestamp, AC/DC power, irradiance, inverter state, quality flag, and meter provenance.

7.5 Phase 3 - Model rebuild and gated RL (Weeks 7–10)

- Train regional load with chronological splits and compare against the national baseline.
- Train site-level PV only when measured site truth and configuration are sufficient.
- Calibrate intervals, bias, drift, and seasonal/site performance; produce model cards.
- Build an RL environment from approved data. Estimated PV may support research but can never make the policy production-ready.
- Compare RL with no-action and deterministic baselines; require zero safety violations.
- Promote agent status only when its own evidence and acceptance gate passes.

7.6 Phase 4 - Product integration and hardening (Weeks 10–14)

- Complete OIDC/JWT, RBAC, tenant isolation, rate limits, audit, and spend controls.
- Make station selection a shared frontend context that drives every relevant page/card.
- Add alerts, acknowledgment, explanation provenance, and blocked-reason UX.
- Implement one read-only plant connector and a replay environment; no control commands.
- Verify HTTPS, actual deployment topology, backups, restore, CI/CD, metrics, traces, and SLOs.

7.7 Phase 5 - Controlled pilot (Weeks 15–22)

- Run a 1–3-site shadow pilot for at least 2–4 weeks.
- Compare forecast, schedule, measured actual, policy result, and operator decision.
- Review every alert, blocked result, model drift event, and LLM fallback.
- If RL passed its gate, show advice only; record human acceptance/rejection and outcome.
- Conduct incident, provider outage, backup/restore, policy rollback, and security drills.
- Hold a go/extend/stop production-readiness review with accepted residual risks.

8 Timeline and Milestones

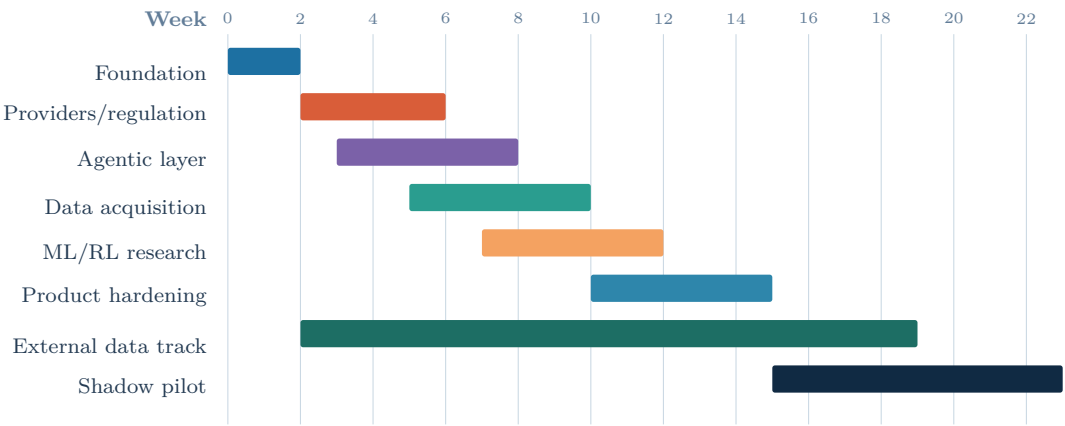


Figure 3: Integrated engineering and controlled-pilot schedule. The 10-week build track is inside the 22-week evidence and pilot track.

Table 6: Milestones and decision gates.

Week	Milestone	Success criterion	Decision
1	M0 Foundation	real-mode audit, registry design, Docker evidence, all tests/build pass	engineering gate
3	M1 Providers/rules	NASA provider validated; KERC source archived and golden cases reviewed	source/legal gate
4	M2 Agentic route	AI route returns schema-valid reasoning and identical numeric output; fallback proven	safety gate
6	M3 Data feasibility	inspected source files; coverage, interval, license, and quality report	data gate
10	M4 Model/RL research	signed model cards; RL beats baseline offline with zero safety breach	research gate
14	M5 Pilot-ready	auth/isolation, UI connection, read-only connector, SLO/restore drills	operational gate
22	M6 Pilot decision	shadow metrics, operator sign-off, incidents and residual risks reviewed	go/extend/stop

9 Standard Implementation Workflow

Every roadmap item follows the same evidence chain:

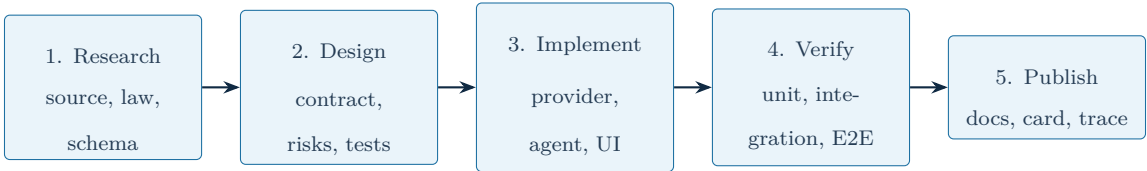


Figure 4: Required implementation workflow for every change.

10 Testing and Acceptance

- **Regression:** current 118 tests, Ruff, and frontend build remain green.
- **Provider contract:** fixtures, timezones, units, missing data, retries, cache, drift.

- **Policy:** official-source golden cases, boundaries, cumulative slabs, cap, rollback.
- **Agentic:** tool allowlist, prompt injection, structured output, numeric equivalence, deterministic fallback, checkpoint/resume, human interrupt, spend limit.
- **Data quality:** coverage, freshness, duplicates, gaps, license, geography, domain shift.
- **ML:** chronological holdout, baseline comparison, calibration, season/site slices, drift.
- **RL:** offline only, deterministic baseline, stress cases, constraints, no unsafe action.
- **Frontend/backend:** contract tests for all client functions and page workflows.
- **Security/operations:** auth, tenant isolation, secrets, rate limits, HTTPS, restore, provider outage, rollback, load, SLO and incident drills.

11 Cost and Capacity Planning

Do not hardcode current model prices or model IDs in application logic. Select models from a capability policy that requires tool calling, structured output, latency, geography/privacy, availability, and budget. Record the actual model returned by the provider.

For planning:

$$\text{daily LLM cost} = \sum_a \left(\text{calls}_a \times \frac{\text{input tokens}_a \times p_{in,a} + \text{output tokens}_a \times p_{out,a}}{10^6} \right)$$

The supplied usage assumption of roughly 50 orchestrations, 50 explanations, and 24 anomaly reviews per day is a **pilot scenario**, not measured traffic. Add cache-hit rate, retries, provider fallback, and a 30–50% contingency. Set daily and monthly hard budgets; when exhausted, fall back to deterministic output.

12 Risks and Honest End State

Table 7: Principal execution risks.

Risk	Effect	Response
Official KERC text or interpretation changes	wrong monetary output	effective-dated rules, archived source, legal review, no-money default, rollback
NPP files lack required load granularity	load pipeline blocked	inspect first; obtain SLDC/site agreement; do not interpolate false hourly truth
Capacity dataset is planned work, not commissioned state	wrong station loading	source classification, commissioning evidence, confidence and manual review
Local PV partnership fails	no production PV/RL validation	retain estimate label; continue decision-support pilot without production RL
LLM changes numbers or invents evidence	unsafe decisions	typed tools, immutable outputs, strict JSON, equivalence tests, deterministic fallback
Prompt injection or secret leakage	data/security incident	allowlists, input separation, no secret in prompts, output validation, audit
Provider/model price or availability changes	cost/outage	model policy, pinned tested options, fallback list, budgets, cache
Frontend and backend contracts diverge	broken operator workflow	generated schemas, contract tests, one shared station context, CI build gate
Managed AWS is assumed deployed from Terraform	false readiness claim	verify actual topology; label code-only infrastructure until observed

After the plan, the system may still honestly report the following as pending: real-time KPTCL/SLDC telemetry without an agreement, on-site pyranometer validation, full local PV and load coverage, inter-state CERC settlement handling, autonomous control, and production RL. These are not documentation failures; they are evidence gates.

13 Immediate Team Actions

1. Approve this architecture and make it the source of truth for implementation.
2. Run the route-by-route APP_DATA_MODE and unsupported-currency audits.
3. Verify Docker Compose and record container evidence.
4. Implement NASA POWER as a historical/NRT fallback with explicit freshness semantics.
5. Archive and legally review the official KERC 2026 source before enabling any rupee output.
6. Create the typed LangGraph state/tool contracts and a deterministic-equivalence test first.
7. Inspect candidate NPP and KPTCL/Aikosh files before committing to schemas or schedules.
8. Send local PV data requests immediately because this is the longest external dependency.

14 External Technical References

- GitHub repository: <https://github.com/DBIT-Bangalore/SuryaGrid>
- LangGraph: <https://github.com/langchain-ai/langgraph>
- OpenRouter API: <https://openrouter.ai/docs/api/reference/overview>
- OpenRouter tool calling: <https://openrouter.ai/docs/guides/features/tool-calling>
- NASA POWER temporal API: <https://power.larc.nasa.gov/docs/services/api/temporal/>
- National Power Portal published reports: <https://www.npp.gov.in/publishedReports>
- Karnataka Electricity Regulatory Commission: <https://kerc.karnataka.gov.in/>